

RAG 知识库检索增强生成系统

工程设计文档 — 方向 B：智能客户支持与检索增强生成助手

1. 执行摘要

业务问题：企业员工每天在数百份非结构化文档中查找答案，传统关键词搜索无法理解'交作业'与'提交方式'的语义等价关系，导致重复提问率高、信息检索效率低下。

为什么传统方法做不到：关键词搜索依赖字面匹配——用户搜'怎么交作业'，文档中写的是'提交方式'，返回零结果。当文档规模超过数百份且表述方式多样时，基于 TF-IDF 或 BM25 的检索召回率急剧下降。

核心交付：我们构建了一个端到端 RAG 系统，从 176 篇杂乱文档自动构建可搜索向量索引。系统在 41 个域内查询上实现 87.8% 的 Recall@3，端到端稳态延迟 6.2 秒，建库成本低于 1 元人民币。

2. 系统架构

系统采用七层流水线架构，每层职责单一、单向依赖：ingest.py（摄取）→ clean_text()（清洗）→ chunk_text()（分块：段落→句子→贪心→滑动）→ extract_metadata()（LLM 元数据）→ Qwen3-Embedding-0.6B（本地 GPU，1024 维）→ ChromaDB（HNSW，cosine）→ query_parser.py（意图解析）→ qa.py（答案生成）。

架构设计原则：(1) 模块化——每个文件职责单一，杜绝上帝脚本；(2) 路径可移植——所有路径基于 BASE_DIR 相对定位，无硬编码绝对路径；(3) 环境变量——API Key 等敏感配置通过 .env 管理；(4) 单例模式——OpenAI 客户端全局缓存复用；(5) 降级设计——查询解析失败→全文搜索，where 过滤失败→纯语义搜索。

2.1 技术栈

层级	组件	技术选择
文档摄取	PDF/MD/TXT	PyMuPDF + PyYAML
文本清洗	正则流水线	Python re
语义分块	四层算法	自研（700字符，120重叠）
元数据	LLM 结构化	DeepSeek V4 Flash（32线程）
向量嵌入	本地 GPU	Qwen3-Embedding（1024维）
向量数据库	HNSW 索引	ChromaDB（cosine）
查询解析	LLM 意图	DeepSeek V4 Flash
答案生成	LLM+来源	DeepSeek V4 Flash
前端	Web 界面	Streamlit + 自定义 CSS

2.2 模块职责

模块	职责
ingest.py	递归读取 .md/.txt/.pdf，解析 Front-Ma
collect_*.py	Wikipedia/SO/CSDN 三源 API 采集
preprocess.py	清洗+分块+元数据提取（431行）
embed_store.py	ChromaDB 封装：嵌入、检索、安全删除
query_parser.py	自然语言 → {search_query, filters}
qa.py	System Prompt 约束 + 来源强制标注
main.py	CLI 子命令编排
streamlit_app.py	Web UI + 调试模式 + 数据管理

2.3 数据生命周期追踪

以一个具体查询为例，追踪数据在系统中的完整生命周期：用户问 '2025年的通知有哪些？'（1）query_parser.py 调用 LLM，输出 {search_query: '通知', filters: {year:2025, category:'notice'}}；（2）embed_store.py 对'通知'做向量嵌入（Qwen3，1024 维，50ms）；（3）ChromaDB 在 481 个文档块上执行 HNSW近似最近邻搜索，结合 where 过滤，返回 Top-3 结果；（4）qa.py 将检索到的3 个块拼接为约 2000 token 的上下文 System Prompt；（5）DeepSeek V4 Flash 生成带来源标注的答案。全过程端到端 6.2 秒，其中 LLM 调用占 96%。

2.4 '玻璃盒'中间结果展示

系统在 Streamlit 界面提供'调试模式'，展示完整的中间结果：查询解析器输出的 search_query 和 filters、每个检索结果的余弦距离分数（如 0.162，0.246，0.287）、文档来源文件名、以及截断到 400 字的内容片段。用户可直观看到 LLM 基于什么证据生成答案，从而判断答案可信度。调试模式还可实时调节 Top-K（1-10）和 max_distance 阈值（0.1-2.0），展示精度/召回权衡的实时效果。

3. 设计决策与取舍

3.1 向量数据库：ChromaDB vs Milvus vs Qdrant vs Pinecone

方案	优势	劣势	决策
ChromaDB	pip install 即用	不支持分布式	选择
Milvus	生产级分布式	需 Docker+etcd	过度工程化
Qdrant	Rust 高性能	需独立服务进程	部署成本高
Pinecone	全托管	付费 SaaS	数据安全风险

选择 ChromaDB 的理由：课程场景无并发/分布式需求，优先降低部署成本。已做抽象封装，未来可平滑迁移。已知代价：复合 where 条件需显式 \$and 操作符。

3.2 分块策略：四层语义分块

我们选择了四层优先级语义分块算法（段落→句子→贪心合并→滑动窗口切割），而非简单固定长度切分。chunk_size=700，overlap=120。

方案	优势	劣势	Recall@3
固定长度(500字符)	实现快	破坏语义完整性	62%
四层语义分块	尊重边界	实现复杂度高	90%
标题感知分块	利用文档结构	依赖格式规范化	85%

700 字符选择依据：小于 400 导致语义碎片化，大于 1000 稀释向量聚焦度。700 约合 350 中文字，平衡精度与完整性。overlap=120 确保边界句子在相邻块中都有完整表示。四层算法的详细工作流程：第一层检查是否超出 chunk_size；第二层将段落按标点切为句子；第三层贪心合并；第四层滑动窗口强制切割。

3.3 嵌入模型：Qwen3-Embedding-0.6B

方案	维度	成本	延迟	中文
Qwen3-0.6B	1024	免费 (GPU)	50ms/条	优秀
OpenAI ada-002	1536	\$0.0001/1K	200ms/条	良好
MiniLM-L6-v2	384	免费	30ms/条	一般

选择理由：（1）本地 GPU 推理，零 API 成本，建库 450+

块嵌入费用为零；(2) Qwen3 原生中文语义理解优于英文为主的 MiniLM；(3) 1024 维区分度优于 384 维。已知代价：首次加载约 16 秒冷启动。Embedding 模型的选择直接影响检索质量——若模型不理解中文语义，整个 RAG 系统的基础就垮了。

3.4 LLM 选择：DeepSeek V4 Flash

DeepSeek 与 GPT-4o-mini 同价位（约 \$0.15/百万 token），国内网络访问更稳定。本地 7B 模型在 CPU 上延迟超过 10 秒，不适合交互场景。LLM 在本系统中承担三个角色：(1) 元数据提取——从文档前 1200 字符提取作者/年份/分类/摘要，32 线程并发，429 限流自动指数退避重试；(2) 查询解析——自然语言转结构化搜索参数；(3) 答案生成——基于检索上下文生成带来源标注的答案。

3.5 查询解析：LLM 意图提取

输入: "2024年的通知讲了啥"
输出: {search_query: "通知", filters: {year:2024, category:"notice"}}

回退策略: JSON 解析异常时, search_query 回退为原始问题, filters=None, 降级为纯语义搜索。评估中 3/50 个查询触发了此回退。

3.6 失败尝试：正则表达式分块 vs 语义分块

我们最初尝试用正则表达式将文档切为单句，直接按字符数累积到 chunk_size=700。这一方案在英文文档上效果良好，但在中文课程文档上出现严重问题：(1) 中文句中常含英文代码块，正则无法识别代码边界，代码被截断在块中间；(2) FAQ 短文本被切为两个独立块，语义关系丢失。结果：在 20 个中文 FAQ 查询中，正则方案 Recall@3 仅 62%，而四层语义方案达到 90%。代价：代码复杂度从 20 行增至 80 行，但召回率提升 28 个百分点。教训：纯正则忽略文档类型差异，递进式分块策略能对不同类型的文档自适应。

4. 评估与失效模式

4.1 评估方法

构建 50 个测试查询，覆盖 5 类：课程信息(10)、技术概念(10)、跨文档(10)、元数据过滤(10)、边界/超纲(10)。评估三个维度：(1) 检索命中率——正确答案是否出现在 Top-3 结果中；(2) 答案相关性——人工 1-5 分评分；(3) 幻觉率——超纲查询中 LLM 编造答案的频率。

4.2 评估结果

指标	数值	说明
Recall@3	87.8% (36/41)	域内查询 Top-3 检索命中率
人工评分均值	4.36 / 5	50 个查询的人工评分
幻觉率	1/9 (11.1%)	超纲查询中编造答案比例
平均延迟	5.6s (稳态 6.2s)	含首次冷启动
解析成功率	94% (47/50)	3 次 JSON 解析失败自动回退

Recall@3 分类别表现: 课程信息 90% (9/10) ; 技术概念 100% (10/10) ; 跨文档综合 100% (10/10) ; 元数据过滤 70% (7/10) 。

4.3 典型问答示例

示例 1 (技术概念, 5 分): 问'什么是 RAG?'——系统检索到

wiki_检索增强生成.md (distance=0.162) 和 rag_system_notes.md (0.246)，答案准确引用两份来源。

示例 2 (跨文档, 5 分): 问'Hadoop 和 Spark 有什么区别?'——检索到

csdn_spark、wiki_apache_spark、wiki_hadoop 三个来源, 答案对比了内存计算 vs 磁盘 I/O 的差异。

示例 3 (元数据过滤, 4 分): 问'2025年的通知?'——查询解析器提取了 filters, 但 ChromaDB 复合 where 报错回退为纯语义搜索, 返回了通用通知而非精确匹配。

示例 4 (超纲幻觉, 1 分): 问'Python for 循环怎么写?'——检索到 SO 上的 pandas 代码片段, LLM 基于代码给出了详细示例。系统未拒答, 属于语义漂移幻觉。

4.4 失效模式分析

失效 1: 复合 where 条件不兼容。Q31 触发 ChromaDB 报错, 回退为纯语义搜索。根因: ChromaDB 不支持隐式 AND。已修复: 自动转为 \$and 格式。

失效 2: 超纲查询幻觉。Q44 检索到 SO 代码片段, LLM 给出详细代码——语义漂移。修复方向: System Prompt 增加主题相关性约束。

失效 3: 查询解析 JSON 格式错误。Q32/Q38/Q39 返回非标准 JSON, 正确回退但丢失过滤能力。修复方向: 增加 JSON 修复逻辑。

4.5 事后剖析: 两个重大失败

失败 1: SentenceTransformer 方法名变更导致全链路崩溃。commit 92bc9b2 将 get_sentence_embedding_dimension() 改为 get_embedding_dimension(), 基于 PyTorch deprecation 警告。但当前版本仍用旧方法名。单元测试因 Mock 了整个模型未发现。教训: Mock 测试无法发现 API 兼容性问题, 需集成冒烟测试。

失败 2: 极短文档静默丢弃。早期 clean_text() 对小于 10 字符文档直接丢弃, 导致 FAQ 短答案丢失。修复: chunk_text() 末尾兜底保留非空极短文档。教训: 清洗应区分'噪声'和'短但有效'的内容。

4.6 避免的反模式

(1) 上帝脚本——保持 11 个模块各司其职; (2) 硬编码路径——基于 BASE_DIR 推导; (3) print 替代日志——统一使用 get_logger(); (4) 静默吞异常——所有 except 至少 logger.warning(), 且保留 KeyboardInterrupt 信号穿透; (5) 先写 UI 后写引擎——遵循'行走的骨架'策略, 第三天就让一行数据走通全流水线。

4.7 安全与 Prompt Injection 讨论

当前系统的安全边界: (1) System Prompt 限定 LLM 仅基于参考资料回答, 但未做 prompt injection 防御。修复方向: 输入清洗阶段增加指令模式检测。(2) XSS 防护——前端对 LLM 输出做 html.escape() 转义。(3) API Key 管理——通过 .env 文件隔离, 不硬编码在代码中。

5. 延迟与成本估算

5.1 延迟预算

组件	首次查询	稳态查询	占比
模型加载(冷启动)	16.0s	0s	—
查询解析(LLM)	2.2s	2.7s	43%
向量检索	16.2s*	0.25s	4%
答案生成(LLM)	3.5s	3.2s	53%
端到端	22.0s	6.2s	100%

*首次检索含嵌入模型加载时间。瓶颈：LLM 调用占稳态延迟 96%，向量检索仅 250ms。优化方向：(1) 缓存高频查询解析结果；(2) 换更小 LLM 做解析；(3) 异步并行查询解析与嵌入计算；(4) 答案生成用流式输出改善感知延迟。

5.2 课程项目实际成本

调用类型	次数	单价	总成本
元数据提取(LLM)	176次	0.005	0.88
查询解析(LLM)	每次	0.005	按量
答案生成(LLM)	每次	0.01	按量
文本嵌入(本地)	450次	0	0
建库总成本	—	—	< 1.0
单次查询成本	—	—	~ 0.015
每1000次查询	—	—	~ 15

5.3 每 1000 次查询成本详解

组件	单次	1000次	备注
查询解析(LLM)	0.005	5	~500 in+50 out tokens
答案生成(LLM)	0.01	10	~2000 ctx+500 out
向量嵌入(本地)	0	0	Qwen3 GPU 免费推理
合计	0.015	15	

对比：若使用 OpenAI ada-002 做嵌入加 GPT-4o-mini 生成，每 1000 次约 50-80 元。本地嵌入节省 70% 查询成本。在 10TB/天的云规模下，Embedding 本地化每月可节省 15,000-40,000 元的 API 调用费。

5.4 云成本估算（10TB/天规模）

类别	月估算	依据
计算(ECS)	30K-60K	20×ecs.g6.4xlarge, 1.5/h
存储(OSS)	10K-20K	10TB/天×30×0.12/GB
模型调用	20K-80K	LLM按查询量
网络传输	5K-10K	跨可用区数据传输
合计	65K-170K	

5.5 可扩展性讨论

若数据量从 176 篇扩展到 10,000+ 篇：(1) 向量索引从 ChromaDB 迁移到 Milvus/Qdrant，支持分布式索引；(2) ETL 从纯 Python 切换到 PySpark；(3) Embedding 批量计算使用 GPU 集群；(4) LLM 调用增加 Redis 缓存层。当前架构的抽象层设计使得数据库迁移只需修改 embed_store.py 一个文件。

6. 附录

6.1 运行方式

pip install -r requirements.txt
copy .env.example .env # 填入 OPENAI_API_KEY
python src/main.py collect-all
python src/main.py build
python src/main.py ask --question "课程项目提交要求是什么?"
streamlit run app/streamlit_app.py

```
python -m pytest tests/ -v
```

6.2 演示方案（15 分钟）

环节	时长	内容
开场钩子	1min	业务问题引入
现场演示	3-4min	Streamlit 端到端问答
架构深潜	4min	数据生命周期追踪
我们搞砸了	2min	失败案例分享
Q&A	5min	技术总监提问

6.3 Q&A 预备问答

Q1: 检索耗时 2 秒，瓶颈在哪里？A: 瓶颈是 LLM 调用（查询解析 2.7s + 生成 3.2s），向量检索仅 250ms。优化：缓存解析结果、换更小 LLM、异步并行。

Q2: 如果攻击者在源文档注入恶意指令？A: 当前 System Prompt 已限定仅基于参考资料回答，但未做 prompt injection 防御。后续可在输入清洗阶段增加指令过滤。

Q3: 为什么用 LLM 而不用关键词匹配？A: 关键词搜索无法理解'交作业'等于'提交方式'的语义等价。向量检索在语义理解上有根本优势，但代价是 6.2s 延迟。